



Khulna University of Engineering & Technology

Department of Computer Science and Engineering

Meowtropolis Project Report

A SwiftUI + Firebase Pet Care Application

Course No:

CSE 3218

Course Title:

Mobile Computing Laboratory

Submission Date:

April 19, 2026

Submitted By

Tawhidul Hasan

Roll: 2107004

Md. Saif Ahmed Shejan

Roll: 2107009

Md. Nafiz Ahmed

Roll: 2107001

Submitted To

Md. Repon Islam

Assistant Professor

Dept. of CSE, KUET

Most. Kaniz Fatema Isha

Lecturer

Dept. of CSE, KUET

1. Objectives

The primary objectives of the Meowtropolis project are:

- To design and develop a comprehensive, user friendly iOS pet care application using SwiftUI and Firebase as the core technology stack.
- To implement secure and seamless user authentication, including sign up, login, password reset, and session management.
- To enable pet owners to create and manage detailed pet profiles with rich information.
- To integrate grooming and veterinary appointment booking functionalities within a single unified platform.
- To build a fully functional marketplace with product listings, search, filtering, cart management, and safe checkout flow.
- To provide a curated pet-focused blog section to support pet education and engagement.
- To apply software engineering best practices including clean architecture, separation of concerns, and layered design.
- To gain hands-on practical experience with mobile computing paradigms, real-time databases, and cloud-backed iOS development.

2. Introduction

Meowtropolis is a feature-rich iOS application designed to serve as an all-in-one digital companion for pet owners. The application was developed as part of the CSE 3218 Mobile Computing Laboratory course, aiming to bridge the gap between everyday pet care needs and modern mobile technology.

In today's fast-paced environment, pet owners frequently juggle multiple responsibilities: finding veterinary care, scheduling grooming appointments, purchasing pet supplies, and staying informed about animal health. Meowtropolis consolidates all these workflows into a single, cohesive mobile experience.

The application is built on a robust technology stack. The frontend is developed entirely in Swift and SwiftUI, Apple's declarative UI framework, which enables reactive and performant user interfaces. The backend is powered by Firebase, specifically Firebase Authentication for identity management and Cloud Firestore as a scalable, real-time NoSQL database.

This app also features a built-in product seeding mechanism that auto-populates the marketplace from a bundled JSON file when the Firestore products collection is empty, ensuring a smooth first-run experience.

From an architectural perspective, Meowtropolis adheres to a clean layered structure separating Models, Services, State, and Views. This promotes maintainability, testability, and scalability. Both unit tests (XCTest) and UI tests are included in the project, providing a validation layer for critical application flows.

3. Class Design and Behavior

Meowtropolis is organized into four primary architectural layers, each containing purpose-specific classes and structures. This section describes the major classes, their responsibilities, and how they interact within the application.

3.1 Models Layer

The Models layer defines the Codable data structures that represent the core domain entities of the application. These are lightweight, framework-independent Swift structs and classes.

- **User:** Represents an authenticated user with fields for UID, email, display name, and profile image URL. Conforms to Codable for Firestore serialization.
- **Pet:** Encapsulates pet profile data including name, species, breed, age, and owner reference. Supports CRUD operations through the PetService.
- **Product:** Models marketplace items with fields such as id, name, price, category, imageURL, and an optional stock count. Includes default handling for missing stock values.
- **Order:** Captures a placed order with cart snapshot, total amount, timestamp, and user reference. Used in the checkout flow.
- **Booking and VetRequest:** Domain models for grooming appointments and veterinary consultations respectively, each containing scheduling metadata and status fields.

3.2 Services Layer

The Services layer encapsulates all side effects and external integrations. Each service is a singleton or injected dependency that provides async methods callable from State classes.

- **AuthService:** Wraps Firebase Authentication. Handles sign-up, login, logout, password reset, reauthentication, and account deletion. Publishes a `@Published currentUser` property consumed by State.
- **UserService:** Manages reading and writing user profile data to Firestore. Handles profile image compression, upload to base64 Firestore storage, and personal information updates.
- **PetService:** Provides create, read, update, delete operations for pet documents under the `users/{uid}/pets` Firestore subcollection.
- **ProductService:** Fetches product listings from Firestore with search and sort support. Delegates first-run seeding to ProductSeedService.
- **ProductSeedService:** Reads `SampleData/products.json` from the app bundle and uploads entries to Firestore when the products collection is empty. A local flag prevents repeated uploads.
- **OrderService:** Handles stock-safe order placement using Firestore transactions to decrement product stock atomically before writing the order document.
- **BookingService and VetRequestService:** Submit booking and vet request documents to their respective Firestore collections.

- **MapsService:** Wraps the Google Places SDK. Constructs and dispatches nearby place search queries, parses results, and handles configuration and API key errors.

3.3 State Layer

The State layer contains `ObservableObject` classes that serve as view models. They orchestrate data flow, call Services, and expose Published properties that SwiftUI views observe and react to.

- **AuthState:** Manages the authentication lifecycle — tracking the signed-in user, routing to the correct root view, and surfacing human-readable auth errors.
- **ProfileState:** Handles account management operations — image upload, name updates, password changes, account deletion — with loading and error state exposure.
- **PetState:** Owns the list of user pets and exposes actions for adding, editing, and deleting pets with reactive UI updates.
- **ProductState:** Manages product fetch, search query binding, active sort/filter selections, and pagination state.
- **CartState:** Maintains the in-memory cart, quantity adjustments, savings calculation, and recommendation generation.
- **CheckoutState:** Coordinates the multi-step checkout flow including address input, order review, and order submission via `OrderService`.
- **MapState:** Drives the nearby search experience — managing loading, success, empty, and error states returned from `MapsService`.

3.4 Views Layer

The Views layer contains all SwiftUI views organized into logical subfolders. Views are purely declarative and derive their content from State objects injected as environment objects or observed state.

- **Auth/:** Login, signup, and password reset screens. Consume `AuthState` for form validation and submission.
- **Main/:** Core tab-based navigation. Hosts Pet, Marketplace, Map, and Blog tabs.
- **Common/:** Reusable UI components such as product cards, pet avatars, loading indicators, and error banners.
- **Shared/:** Utility views such as image pickers and form field wrappers shared across multiple screens.

4. System Diagrams

This section presents the key architectural and behavioral diagrams for the Meowtropolis application, including the Entity-Relationship (ER) diagram, Data Flow Diagram (DFD), and Activity Diagram.

4.1 Entity-Relationship (ER) Diagram

The ER diagram illustrates the primary data entities stored in Cloud Firestore and their relationships. Meowtropolis uses a document-based NoSQL model with the following key entities and associations:

Entity	Key Attributes	Relationships
User	uid, email, displayName, photoURL, createdAt	Has many Pets, Bookings, VetRequests, Orders
Pet	id, ownerId, name, species, breed, age, imageURL	Belongs to one User
Booking	id, userId, petId, serviceType, date, status	Belongs to User; references Pet
VetRequest	id, userId, petId, symptoms, scheduledDate, status	Belongs to User; references Pet
Product	id, name, price, category, imageURL, stock	Referenced by OrderItem
Order	id, userId, items[], totalAmount, timestamp	Belongs to User; contains OrderItems referencing Products

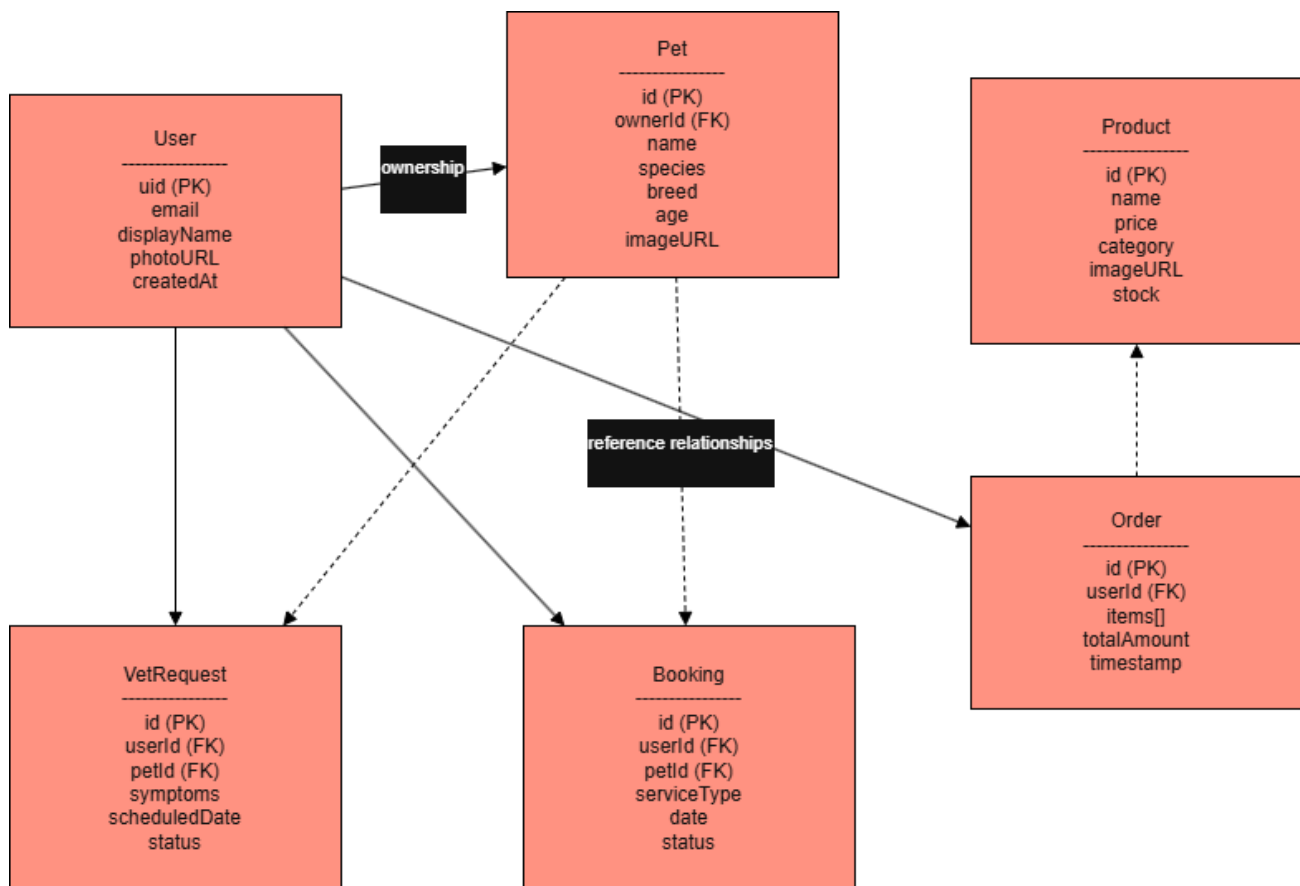


Fig 1: ER Diagram

4.2 Data Flow Diagram (DFD) - Level 0 (Context Diagram)

The Level 0 DFD presents the highest-level view of the system, showing Meowtropolis as a single process with its primary external entities and data flows:

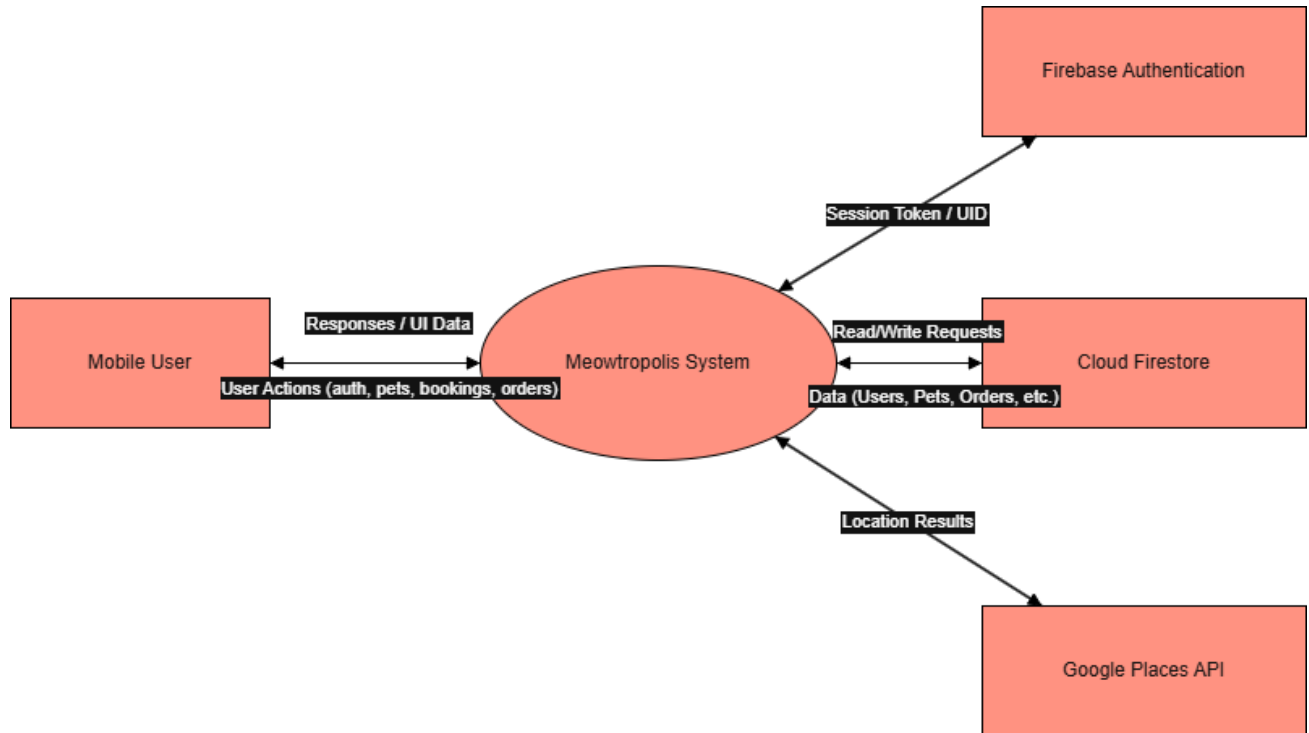


Fig 2: Data Flow Diagram Level 0

4.3 Data Flow Diagram - Level 1

The Level 1 DFD decomposes the Meowtropolis system into its primary sub-processes:

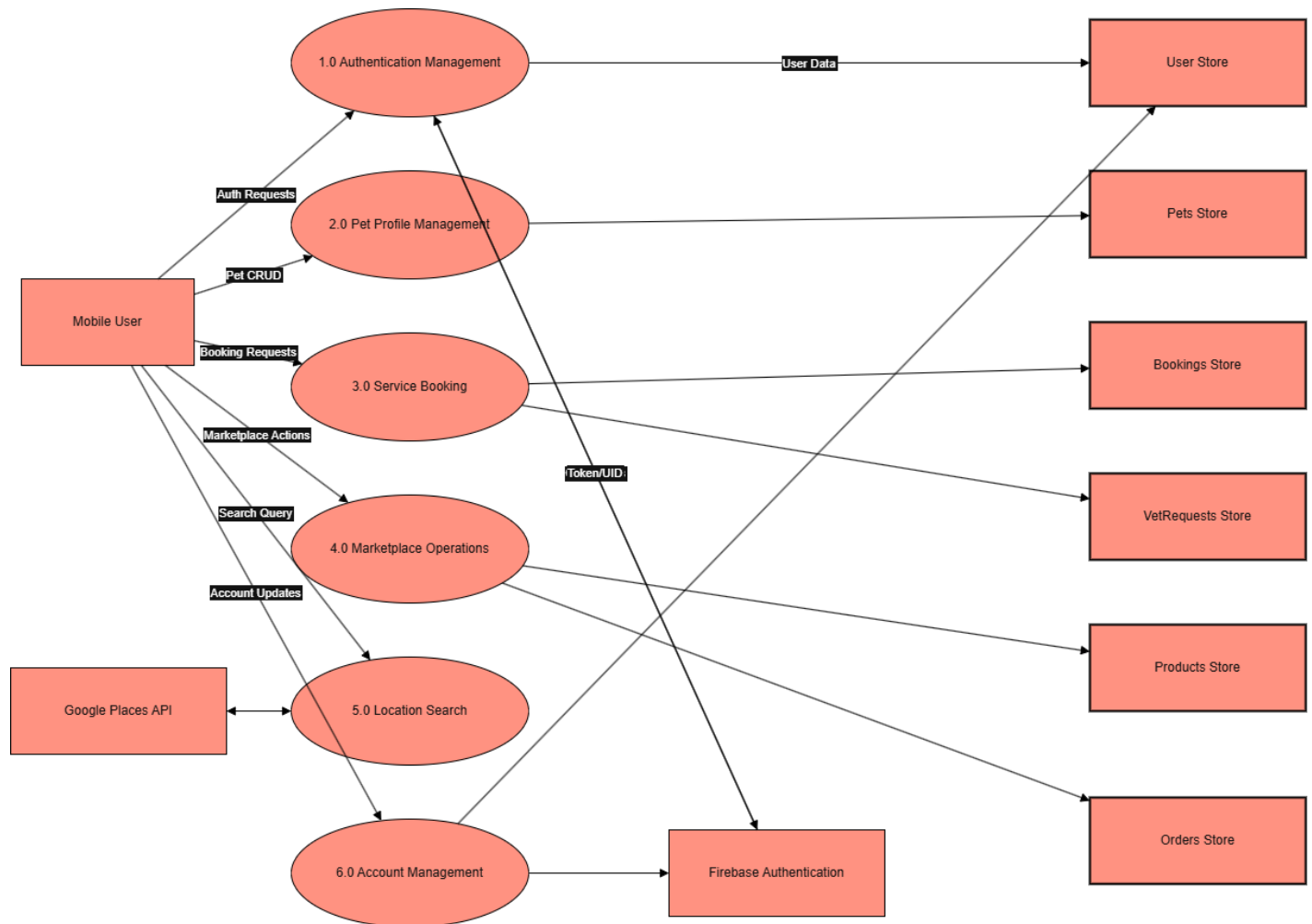


Fig 3: Data Flow Diagram Level 1

4.4 Activity Diagram - Checkout Flow

The activity diagram below describes the step-by-step flow of a user completing a purchase through the Meowtropolis marketplace:

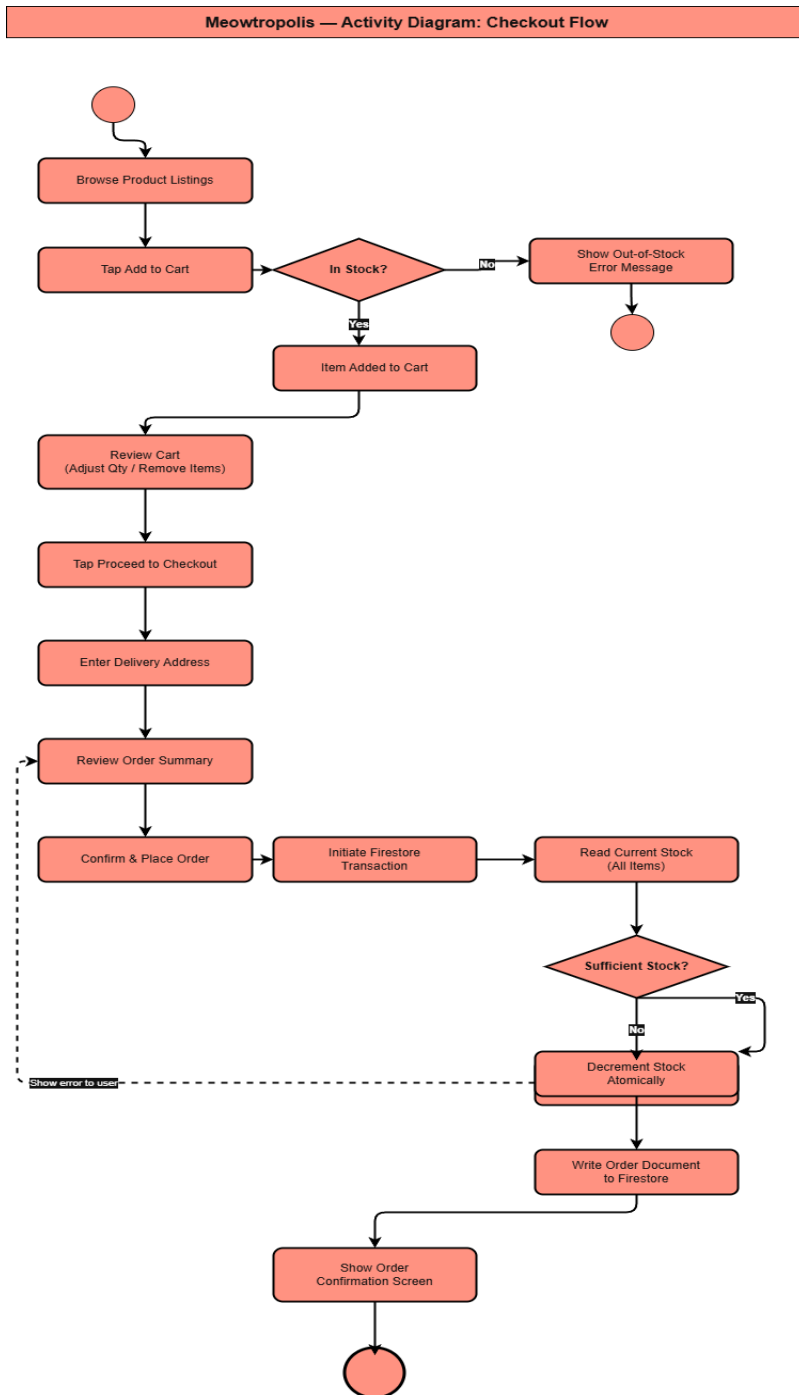


Fig 4: Activity Diagram (Checkout Flow)

5. Discussion

The development of Meowtropolis offered a multifaceted learning experience that spanned frontend UI engineering, backend integration, data modeling, and architectural decision-making. This section discusses the key insights and challenges encountered throughout the project.

5.1 SwiftUI and Reactive Programming

Building the UI with SwiftUI reinforced the power of declarative, reactive programming. The use of `@Published` properties in `ObservableObject` classes, combined with `@StateObject` and `@EnvironmentObject` wrappers in views, enabled clean data-driven UI updates without manual view refresh logic. One challenge encountered was managing state across deeply nested views, which was resolved by passing state objects through the SwiftUI environment.

5.2 Firebase Integration

Firebase proved to be a well-suited backend for a mobile-first application. Cloud Firestore's document model aligned naturally with the app's hierarchical data (e.g., users containing pet subcollections). The real-time listener capabilities of Firestore allowed for instant UI updates when data changed. Managing Firestore security rules for the development baseline and understanding the implications for production hardening was a valuable exercise.

5.3 Atomic Transactions for Order Placement

One of the more technically demanding features was implementing stock-safe order placement. Using Firestore transactions, the application reads the current stock of each ordered product and decrements it atomically before writing the order document. This prevents race conditions where two users could simultaneously order the last item in stock. This exercise deepened understanding of distributed transaction semantics.

5.4 Product Auto-Seeding

The product auto-seeding mechanism using a bundled `products.json` file and a local one-time flag demonstrated a practical approach to bootstrapping an empty database for demo and testing purposes. It also highlighted the importance of idempotency — the seeding logic checks whether the products collection is already populated before uploading, preventing duplicate data.

5.5 Architecture and Separation of Concerns

Adhering to the Models–Services–State–Views layered architecture promoted clean code organization. Keeping side effects isolated in Services and flow logic in State classes made the codebase more testable and maintainable. Refactoring early View-level logic into appropriate State classes was a recurring and valuable practice throughout development.

6. Conclusion

Meowtropolis successfully demonstrates the feasibility of building a production-quality, multi-feature iOS application within an academic project context. By combining SwiftUI's reactive UI paradigm with Firebase's scalable cloud backend and Google's mapping capabilities, the team delivered a cohesive platform addressing real pet owner needs.

The project achieved all defined objectives: a secure authentication system, full pet profile management, integrated service booking, a functional marketplace with atomic order placement, location-aware nearby search, and a content-rich blog section. The clean layered architecture ensured maintainability and provided a solid foundation for future feature expansion.

From an educational standpoint, the project provided hands-on exposure to some of the most widely adopted mobile development tools and patterns in the industry — SwiftUI, Firebase, Firestore transactions, Google Places API, and XCTest. The challenges encountered, including state management complexity, distributed transaction semantics, and third-party SDK configuration, prepared the team for real-world mobile engineering demands.

Future enhancements could include real payment gateway integration (e.g., Stripe), push notifications for booking confirmations and vet reminders, expanded social features for pet owners to share experiences, and transition to a more scalable image storage solution such as Firebase Cloud Storage.

Overall, Meowtropolis stands as a testament to the practical application of mobile computing principles, and the team is proud to present it as a complete, validated MVP ready for demonstration and further development.